

Лабораторная работа №6. Работа с файлами. Списки

Программа должна создавать файл *.xls (xlsx), записать в него сгенерированный случайным образом массив чисел. Затем, с помощью реализованных алгоритмов сортировки – всех, ниже приведенных, записать отсортированную последовательность чисел в ранее созданный файл *.xls. В результирующем файле должны быть: исходный неотсортированный массив чисел и отсортированные разными алгоритмами массивы чисел, столбцы должны иметь заголовки, в которых помимо названия алгоритма должна быть продолжительность работы алгоритма.

Алгоритмы сортировки:

1. Сортировка выбором
2. Сортировка вставками
3. Сортировка "Методом пузырька"
4. Сортировка слиянием
5. Быстрая сортировка

Процесс создания, сохранения и работы с файлами на диске

Для создания файла необходимо выполнить команду:

```
f = open ("file1.txt", "w+")
```

Мы объявили переменную *f*, чтобы открыть файл с именем "file1.txt". Функция *open* принимает 2 аргумента: путь к файлу, который мы хотим открыть, и строку, представляющую виды разрешений или операций, которые мы хотим выполнить над файлом.

Возможные варианты второго аргумента:

"r" открыть текстовый файл для чтения. Поток расположен в начале файла.

"r+" открыт для чтения и письма. Поток расположен в начале файла.

"w" обрезать файл до нулевой длины или создать текстовый файл для записи. Поток располагается в начале файла.

"w+" открыт для чтения и письма. Файл создается, если он не существует, в противном случае он затирается. Поток расположен в начале файла.

"a" открыт для письма. Файл создается, если он не существует. Поток расположен в конце файла. Последующие записи к файлу всегда будут в конце текущего конца файла.

"a+" файл открыт для чтения и письма, если файл не существует, то он создается. Поток располагается в конце файла.

Варианты второго аргумента	r	r+	w	w+	a	a+
Read (чтение из файла)	+	+		+		+
Write (запись в файл)		+	+	+	+	+
Write after seek (возможно начала записи с произвольного участка файла)		+	+	+		
Create (создание файла)			+	+	+	+
Truncate (при открытии файл становится пустым)			+	+		
Position at start (позиция при открытии устанавливается в начало файла)	+	+	+	+		
Position at end (позиция при открытии устанавливается в конец файла)					+	+

В следующем примере в цикле в файл записываются строки «This is line 1,2..., n»:

```
for i in range(10):
    f.write("This is line %d\r\n" % (i+1))
f.close()
```

В цикле `for` задается диапазон 10 чисел. Для записи данных в файл используется функция `write`. В каждой строке в файл записывается строка: «This is a line», затем `%d` (предупреждает, что будет введено целое число), символ перевода каретки на новую строку(`\r\n`) и само значения номера строки `%(i+1)` . Таким образом, в основном мы вводим номер строки, которую пишем, затем помещаем ее в символ возврата каретки и символ новой строки. В конце файл необходимо закрыть (функция `f.close()`).

Если вы попытаетесь записать в файл просто созданный список, то он запишется как строковые данные со скобками. Впоследствии это приведет к затруднению считывания данных. Поэтому лучше использовать указания типов при записи данных в файл. Уточним, как работает типизированный ввод данных.

```
>>> pupil = "Ben"
>>> old = 16
>>> grade = 9.2
>>> f.write ("It's %s, %d. Level: %f" % (pupil, old, grade))
It's Ben, 16. Level: 9.200000
```

В вышеприведенном коде создаются 3 переменные строкового, целого и вещественного типа. Для записи их в файл необходимо предварительно указать, в каком порядке и какого типа переменные будут сохраняться "It's %s, %d. Level: %f" , а затем перечислить сами переменные. Таким образом, буквы `s`, `d`, `f` обозначают типы данных – строку, целое число, вещественное число.

Чтение файла

Для открытия файла в режиме чтения наберите команду:

```
f = open("file1.txt", "r")
```

Можно использовать функцию `mode` в коде, чтобы проверить, находится ли файл в открытом режиме. Если да, мы продолжаем:

```
if f.mode == 'r':
```

Используйте `f.read`, чтобы прочесть данные файла и сохранить их в переменной `content`.

1. Способ вывода – выводим все содержимое файла в переменную `content`

```
#Open the file back and read the contents
```

```
f=open("file1.txt", "r")
```

```
if f.mode == 'r':
```

```
    contents=f.read()
```

```
    print (contents)
```

```
f.close()
```

Для того, чтобы получить из строки список отдельных элементов , можно воспользоваться функцией `split("arg")`, где `arg` – символ разбиения:

```
with open('foo.txt','r')as f:
```

```
    c=f.read().split(" ")
```

```
    print(type(c[0]))
```

Символом разбиения может быть пробел (как в примере), и любой символьный знак, например «,», «/» и др.

2. Способ вывода – поэлементное задание и вывод переменной `x`:

```
f=open("file1.txt", "r")
```

```
#or, readlines reads the individual line into a list
```

```
fl =f.readlines()
```

```
for x in fl:
```

```
    print(x)
```

```
f.close()
```

Основные требования к оформлению работ

1. Каждая программа должна содержать в начале:

```
print('Программа делает ..... (выполняет, находит.....) ').
```

2. Необходима проверка корректности вводимых данных: если данные числовые, то проверить, число ли это; если данные имеют объективные ограничения, например, в году не может быть больше 366 дней, день — целое число и не может быть меньше 1, и т.д.

3. При вводе данных с клавиатуры должен быть корректно поставлен вопрос, исключающий разные толкования; для разъяснения лучше привести пример.

4. Все данные, выводимые на экран, должны удобно и однозначно читаться, сопровождаться разъяснениями, не должны «слипаться» между собой и другими надписями или данными.

5. Нужно стараться, чтобы программный код был оптимален, меньше занимал места, был последователен и лаконичен.